

Milestone 1: Technical Understanding

Priority Inheritance Protocol (Resource Access Protocols)

Nnaemeka Nnachi-Egwu
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany
nnaemeka.nnachi-egwu@stud.hshl.de

I. PROBLEM STATEMENT AND MOTIVATION

Problem: Shared exclusive resources in real-time systems cause *priority inversion*: a high-priority task τ_i can be blocked for an *unbounded* duration by lower-priority tasks that do not even use the contested resource.

Formal setting (Buttazzo §7.3, p. 209):

- n periodic tasks $\{\tau_1, \dots, \tau_n\}$, priority P_i (lower number = higher priority, RM assignment).
- m shared resources $\{R_1, \dots, R_m\}$ protected by binary semaphores $\{S_1, \dots, S_m\}$.
- Mutual exclusion via $\text{wait}(S_k) / \text{signal}(S_k)$.
- Critical section $z_{i,k}$ of τ_i on S_k , duration $\delta_{i,k}$; collection $Z_{i,k}, \sigma_i, \gamma_i$.

Why it matters (Kopetz definition, RTS slide 7): A real-time system is correct iff results are produced at the right *physical instant*, not only logically. Priority inversion can cause τ_1 to miss its deadline even with $U < 1$ (slide 48: “EDF is optimal only when no resource competition”). A missed hard deadline is a *correctness failure*, not a performance degradation.

Scenario (Fig. 7.4, Buttazzo p. 208): τ_3 holds S_R ; τ_1 (high prio) arrives and blocks; τ_2 (medium prio, no S_R) preempts τ_3 — τ_1 blocked while lower-priority τ_2 runs. Inversion window: $[t_3, t_6]$. Unbounded if multiple τ_2 -class tasks present.

II. CORE TECHNICAL IDEA

PIP (**Priority Inheritance Protocol**), Sha, Rajkumar, Lehoczky [SRL90], Buttazzo §7.6, pp. 214–225:

When τ_i blocks on S_k held by τ_j ($P_j < P_i$), τ_j **inherits** τ_i 's priority:

$$p_j(R_k) = \max\{P_j, \max_h\{P_h \mid \tau_h \text{ blocked on } R_k\}\}. \quad (1)$$

Effect: Medium-priority tasks (like τ_2) cannot preempt the boosted τ_j — priority inversion bounded.

Restoration: On $\text{pi_signal}(S_k)$, τ_j reverts to $\max\{P_j, \max_h\{p_h \mid \tau_h \text{ still blocked by } \tau_j\}\}$.

Transitivity (Buttazzo p. 215, Lemma 7.2): If $\tau_3 \rightarrow \tau_2 \rightarrow \tau_1$ in blocking chain, τ_3 inherits P_1 via τ_2 . *Only occurs with nested critical sections.*

III. RELEVANT FORMALISM

A. Notation (§7.3, p. 209)

- P_i : nominal priority; p_i : active priority
- $z_{i,k}, Z_{i,k}$: CS and set of CS of τ_i on S_k
- $\delta_{i,k}$: CS duration; σ_i : # CS of τ_i ; γ_i : set of CS that can block τ_i
- l_i : # lower-prio tasks that can block τ_i ; s_i : # semaphores that can block τ_i
- B_i : worst-case blocking time of τ_i

B. Lemmas and Theorems (all Buttazzo §7.6)

- **Lemma 7.1** (push-through, p. 218): S_k causes push-through blocking to τ_i iff $\exists \tau_h : P_h > P_i$ using S_k AND $\exists \tau_j : P_j < P_i$ using S_k .
- **Lemma 7.2** (transitivity, p. 218): Transitivity occurs only with nested CS.
- **Lemma 7.3** (p. 219): τ_i blocked at most l_i times.
- **Lemma 7.4** (p. 219): τ_i blocked at most s_i times.
- **Theorem 7.2** (p. 219, [SRL90]): τ_i blocked at most $\alpha_i = \min(l_i, s_i)$ CS.
- **Lemma 7.5** (p. 221): $Z_{j,k}$ blocks τ_i only if $P_j < P_i \leq C(S_k)$.

C. Blocking Time Computation (§7.6.3, Eqs. 7.9–7.11)

B_i^l = sum along task dimension; B_i^s = sum along semaphore dimension; $B_i = \min(B_i^l, B_i^s)$.

$$\gamma_i = \{Z_{j,k} \mid P_j < P_i \leq C(S_k)\}.$$

Algorithm (Fig. 7.11): complexity $\mathcal{O}(mn^2)$. **Not tight** (Buttazzo p. 222): B_i^l can sum CS on same semaphore (impossible per Lemma 7.4).

D. Schedulability (§7.9)

Liu-Layland (sufficient, Eq. 7.19):

$$\forall i : \sum_{h < i} \frac{C_h}{T_h} + \frac{C_i + B_i}{T_i} \leq i(2^{1/i} - 1).$$

RTA (exact, Eq. 7.22):

$$R_i = C_i + B_i + \sum_{h < i} \left\lceil \frac{R_i}{T_h} \right\rceil C_h \quad (\text{iterate to fixed point}).$$

IV. ASSUMPTIONS AND SCOPE LIMITATIONS

- **Single processor** (explicit in §7.3, p. 209).
- **Fixed priorities** (RM/DM); PIP not defined for EDF (footnote 2, p. 209; Spuri extension exists but not in Ch. 7).
- **WCET known exactly**: $\delta_{i,k}$ values assumed given. In practice, WCET estimation is itself a hard problem [WCET].
- **Binary semaphores**: single-unit resources only.
- **No deadlock prevention**: see §7.6.5, Fig. 7.13.
- **No chained blocking prevention**: α_i can be > 1 , unlike PCP (Theorem 7.4) which bounds to exactly 1.
- **Transparency assumption**: kernel must implement `pi_wait/pi_signal`; application code unchanged.

V. RUNTIME, MEMORY, AND ALGORITHM ASPECTS

- Blocking computation: $\mathcal{O}(mn^2)$ offline (Fig. 7.11 algorithm).
- Exact algorithm (Rajkumar): $\mathcal{O}(2^n)$ — impractical for large n .
- `pi_wait` at runtime: unbounded chain traversal (bounded in practice, e.g. Linux `RTMUTEX_DEPTH_LIMIT=48`).
- Memory: extra fields in TCB (active priority, blocked-on semaphore, semaphore queue) — $\mathcal{O}(n + m)$ additional state.
- Table 7.5 rates PIP “hard” to implement — the only protocol so rated.

VI. FIVE-PROTOCOL COMPARISON

Property	NPP	HLP	PIP	PCP	SRP
Deadlock-free	✓	✓	–	✓	✓
Chained block	–	–	Yes	–	–
Transparent	✓	–	✓	–	–
Max blockings	1	1	α_i	1	1
Impl. effort	Easy	Easy	Hard	Medium	Easy
Dynamic prio	–	–	–	–	✓

Source: Buttazzo Table 7.5, p. 248.

VII. VERIFIED APPLICATION EXAMPLE

Task set: $\tau_1(C=1, T=6, D=6, \text{prio} = 1, \text{CS} = 1)$, $\tau_2(C=2, T=8, D=8, \text{prio} = 2, \text{no CS})$, $\tau_3(C=4, T=24, D=24, \text{prio} = 3, \text{CS} = 4)$. Semaphore S_R ; ceiling $C(S_R) = P_1$.

Blocking: $B_1 = 4$ (direct), $B_2 = 4$ (push-through, Lemma 7.1), $B_3 = 0$.

RTA: $R_1 = 5 \leq 6$ ✓, $R_2 = 8 \leq 8$ ✓, $R_3 = 8 \leq 24$ ✓. All schedulable.

Trace highlight: τ_3 inherits P_1 at $t = 2$; τ_2 push-through blocked; τ_3 CS ends $t = 4$; τ_1 blocking = 2 (within $B_1 = 4$ bound).

Open questions noted during study: (1) $\delta-1$ vs. continuous time: Eq. (7.9) subtracts 1 (discrete artefact); hand trace uses full δ — both valid in their respective contexts. (2) Eq. (7.11) not tight (Buttazzo p. 222); Rajkumar exact algorithm ($\mathcal{O}(2^n)$) beyond scope here. (3) Linux `PTHREAD_PRIO_INHERIT` bounds chain depth at 48 hops — formal guarantees may not fully hold on the implementation. (4) PIP uniprocessor only — exact multicore failure mode addressed in M2.